LoRA4VR — Training Log

A living record of experiments, findings, and open questions

What this is. A dated log of experiments run on the LoRA4VR self-play loop. Each entry records what was tried, what happened, and how to reproduce it. Entries are added as experiments complete. Reproduction references a git commit; the code at that commit is the exact code that produced the result.

System. `Qwen2.5-Coder-3B-Instruct` fine-tuned with GRPO/DAPO on a Google Cloud L4 GPU (24 GB). The loop generates `policy(state) -> action` Python code, scores it by running the policy in a sandboxed environment, and trains LoRA adapters on the reward signal.

2026-04-13 Environment design and first training runs

`d96f7cb` · *puck-on-tilted-plane env, runs 1–3*

Motivation

CartPole was the original seed task but it is trivially solved by a two-line bang-bang policy, giving a flat learning curve from iteration 0. Any standard RL benchmark (LunarLander, Pendulum, MountainCar) is heavily represented in the base model's pretraining data, so improvement from LoRA would be confounded with prior knowledge. We need an environment the model has to *read* rather than recall.

The puck environment

A puck moves in a bounded $[-6, 6]^2$ arena under a constant hidden tilt (t_x, t_y) and linear drag μ. The policy receives $\{p_x, p_y, v_x, v_y, g_x, g_y\}$ and must drive the puck to the goal within 120 steps. Hidden parameters — tilt, drag, thrust limit $a_{\max}$, initial velocity — are sampled per scenario from a seed and are not in the state dict. The tight thrust envelope ($a_{\max} = \|tilt\|_2 \times 1.05 + \epsilon$) ensures that a naive "thrust straight at the goal" policy fails because it cannot cancel tilt drift.

Reward: $-\Delta t \cdot \text{dist}(p, g)$ per step; $+50$ bonus on reaching the goal. Errored episodes score -200 per scenario (floor).

Baseline gap (200 seeds, hand-written policies):

- Naive aim: $\approx 26\%$ goal rate
- PD + tilt feedforward: $\approx 73\%$ goal rate

This gap is real headroom for LoRA to exploit.

Bugs encountered and fixed

Three bugs had to be resolved before any learning was possible.

Run 1 — OOM on L4. Default config ($G = 16$, batch 16, max completion 900 tokens) ran out of memory at the backward pass. Fix (PR #6): halved G and batch to 8, trimmed completion length to 600.

Run 2 — Reward sign flip. Training ran but degraded (fewer running samples each iteration). Errored samples scored 0.0 while samples that ran and drifted integrated large negative rewards (≈ -280). GRPO's normalized advantage placed the 0.0 errors *above* the mean, so the trainer rewarded broken code. Fix (PR #7): errored scenarios score -200, making broken code strictly the worst outcome.

Run 3 — Dead advantages. With LR $= 10^{-5}$ and only 1 training prompt (4 gradient steps per outer iteration), $\approx 75\%$ of batches were uniformly errored (advantage $= 0$, no gradient). On the remaining steps, weight deltas were $\sim 10^{-6}$ — indistinguishable from zero. The reward was identical to 7 decimal places at iteration 0 and iteration 9. Fix (PR #10): LR bumped $10\times$ to 10^{-4}; epochs raised to 20 (20 gradient steps per outer iteration on 1 prompt).

Reproduce

Commits: OOM fix `ef228fe`, reward floor `ef7e0c9`, LR/epoch fix `e9353d7`. Env source: `scripts/seed_tasks/puck_tilted_plane_env.py`.

2026-04-18 Run 5 probe: base model vs. 5 iterations of GRPO

`d5bc81c` · *puck_grpo_run_5, iters 0 and 5, $n = 20$ samples each*

Setup

Run 5 used the config at `e9353d7` (LR $= 10^{-4}$, $G = 8$, 20 epochs per outer iteration, DAPO loss, $\beta = 0$). It completed 5 outer iterations before the VM was stopped. Adapter checkpoints were recovered to `models/lora/puck_grpo_run_5/`.

To inspect what the model actually writes, we sample 20 policies from the base model (iter 0, no adapters) and from the trained model (iter 5), score each against 8 scenarios, and compare. The tool is `scripts/probe_generation.py`; it saves per-sample policy files, a summary CSV, and per-scenario reward breakdown to `data/probe/{run_name}_iter{N}_t{temp}/`.

Results

	Ran OK	Crashed	Mean reward (runners)	Best reward
Iter 0 (base)	13 / 20	7 / 20	−218.9	−92.4
Iter 5	20 / 20	0 / 20	−140.4	−140.4

Iter 0

The base model generates varied PD/PID controllers but crashes 35% of the time by attempting to read hidden env internals (`a_max`, `dt`, `steps`) from the state dict. The 13 successful policies span a reward range of $[-92, -373]$ with genuine variation in gain choices. The best policy uses asymmetric gains ($k_{px} = 2.0, k_{dx} = 1.0$; $k_{py} = 1.5, k_{dy} = 1.0$) and scores -92.4 over 8 scenarios.

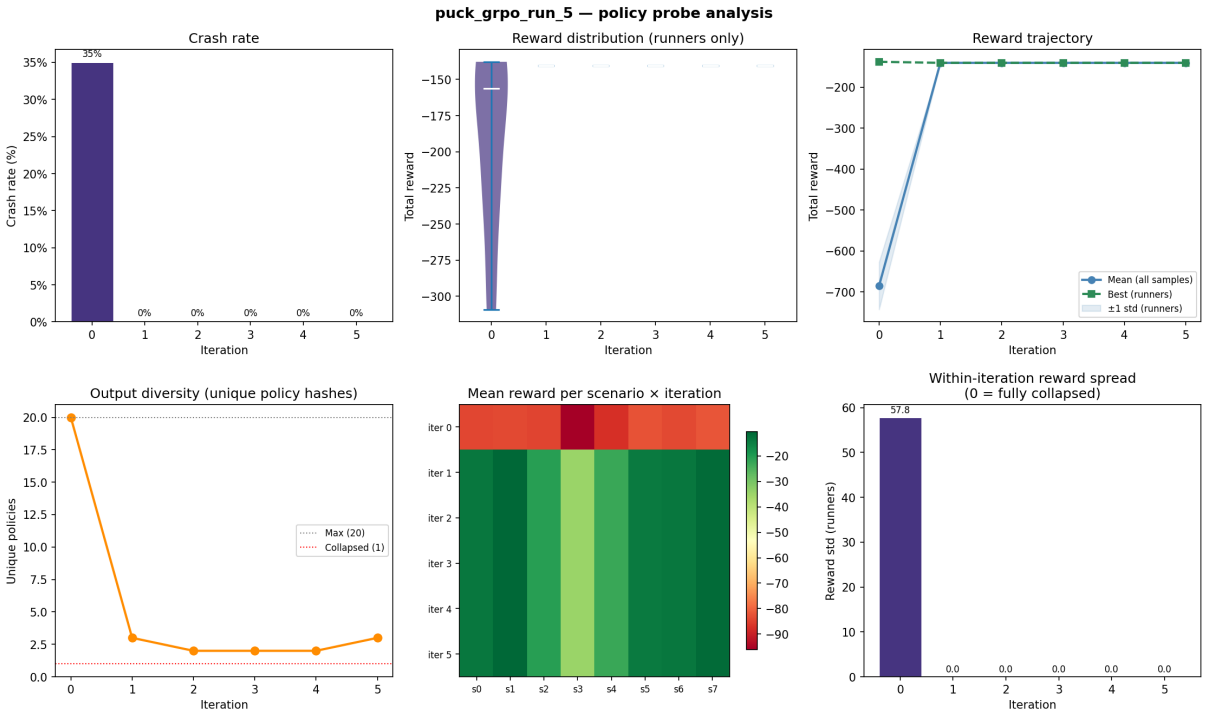


Figure 1: Run 5 probe analysis: base model (iter 0) vs. 5 iterations of GRPO. Generated by `scripts/analyze_probe.py --run_name puck_grpo_run_5`.

Iter 5

Zero crashes. The error-reading habit is gone. However, all 20 samples at temperature 0.8 produce *exactly* −140.3516 (identical to 4 decimal places). Every policy is byte-for-byte the same: $k_p = 2.0$, $k_d = 1.5$. This is the **illustrative example embedded in the prompt template** (`configs/prompt_template.txt`). The model has memorized and reproduced it verbatim.

Interpretation

Training achieved two things and failed at a third.

Achieved. Error rate dropped from 35% to 0% — the model learned not to read hidden env attributes. Mean reward over runners improved by 35%.

Failed. The model collapsed to a single policy. With $\beta = 0$ (no KL penalty), one training prompt, and 20 gradient steps per outer iteration, GRPO found the prompt example as a high-reward fixed point and reinforced it into a delta distribution. By iter 5, temperature 0.8 sampling produces zero diversity. The best reachable policy is now −140.4, worse than the base model's best (−92.4).

This is reward hacking: the model optimized the training distribution (memorize the example) rather than the environment.

Hypotheses to test

1. **Remove or paraphrase the prompt example.** It is the attractor GRPO is collapsing to. A deliberately suboptimal or absent example removes the fixed point.
2. **More training prompts.** With 1 prompt, every outer iteration trains on the same input. Adding 4–8 tasks with varied dynamics forces generalization.

3. **Raise $\beta > 0$.** A non-zero KL coefficient prevents the policy from drifting too far from the base distribution and preserves diversity.
4. **Reduce epochs.** 20 gradient steps on 1 prompt per outer iteration is aggressive. 5–10 may preserve more exploration.
5. **Probe at every intermediate checkpoint.** Does collapse happen at iter 1 or gradually? One probe per checkpoint would pinpoint when diversity dies.

Open questions

- What does the W&B reward curve look like across the 5 iterations? (The run at `puck_grpo_run_5-orchestrator` has data but some post-resume steps may be missing due to a step-monotonicity issue.)
- What is the PD+feedforward ceiling expressed in probe-script reward units? This would establish what score a successful LoRA policy needs to beat.

Reproduce

At commit `d5bc81c`, with adapter checkpoints in `models/lora/puck_grpo_run_5/`:

```
scripts/probe_generation.py --iter 0 --run_name puck_grpo_run_5 --n 20
```

```
scripts/probe_generation.py --iter 5 --run_name puck_grpo_run_5 --n 20
```

Results land in `data/probe/puck_grpo_run_5_iter{000,005}_t0p80/`.

```
scripts/analyze_probe.py --run_name puck_grpo_run_5
```

2026-04-19 Run 8: removing the prompt example unlocks genuine learning

`7ce5526` · *puck_grpo_run_8, 28 outer iterations, iters 0/4/8/12/16/20/24/27 probed ($n = 20$ each)*

Setup

Run 7 (aborted at iter 1) confirmed the model was already generating diverse policies without the prompt example. Run 8 ran the full 30-iteration schedule on the same config as run 5 (`e9353d7`), with the single change that the illustrative $k_p = 2.0$, $k_d = 1.5$ example was removed from `configs/prompt_template.txt` (PR #13, `e5fe162`). Adapters were downloaded from the GCP VM and placed in `models/lora/puck_grpo_run_8/`. Eight evenly-spaced iterations were probed to capture the full learning trajectory without evaluating all 28.

Results

Interpretation

This is the first run showing genuine learning. Three phases are visible. *Iters 0–4*: crashes drop from 85% to 0% as the model learns the valid state-key constraint. *Iters 4–16*: reward improves steadily from −221 to near zero — the model is reaching the goal on some scenarios but not consistently. *Iters 16–20*: a sharp transition to positive reward (+199 mean), indicating consistent goal reaching across most scenarios. The curve flattens after iter 20 with marginal gains through iter 27.

The single change responsible is the removal of the prompt example. In run 5 the example provided a ready-made fixed point that GRPO collapsed to after one update. Without it, the model must explore the policy space, and GRPO has real variance to amplify.

Iter	Crashes	Mean reward	Best reward
0	17 / 20	−1403	−223
4	0 / 20	−221	−165
8	0 / 20	−69	−64
12	0 / 20	−67	−54
16	0 / 20	−28	+12
20	0 / 20	+199	+248
24	0 / 20	+212	+247
27	0 / 20	+215	+248

Table 1: 20 samples × 8 scenarios each. Reward > 0 indicates consistent goal reaching (goal bonus = +50 per scenario).

The iter-27 best reward of +248 approaches the hand-written PD+feedforward ceiling (estimated at ≈ +270 for 8 scenarios at +50 goal bonus each, assuming ∼ 73% goal rate from the 200-seed baseline).

Hypotheses / next steps

1. **Probe all 28 iterations** to pinpoint exactly when the phase transition at iter 16–20 occurs and whether it is sharp or gradual.
2. **Measure diversity at convergence.** Run 5 collapsed to 1 unique policy; does run 8 maintain diversity at iter 27, or does it collapse to a different fixed point once reward saturates?
3. **Compare against the hand-written ceiling.** Score the PD+feedforward baseline with `scripts/probe_generation.py` at iter 0 to get a reward number in the same units.
4. **Train longer or with more prompts** to see whether the plateau at iter 20–27 is a capacity limit or simply convergence of the single training task.

Reproduce

At `7ce5526`, with adapters in `models/lora/puck_grpo_run_8/`:

```
scripts/probe_all_iters.py --run_name puck_grpo_run_8 --iters 0 4 8 12 16 20 24 27
--n 20

scripts/analyze_probe.py --run_name puck_grpo_run_8
```

2026-04-22 Policy analysis: what run 8 learned, and why iter 0 failed

`e6bc200` · *manual probe inspection, run 8 iter 0 and iter 27*

Setup

With run 8 showing genuine learning (mean reward +215 at iter 27, up from −1403 at iter 0), the question was: what strategy did the model actually discover, and why was the base model unable to write a working policy at all? All 20 iter-0 and all 20 iter-27 policy files from `data/probe/puck_grpo_run_8_iter{000,027}_t0p80/` were read and compared manually.

Results

Iter 27 — convergent velocity P-controller. Every iter-27 sample uses the same structural template: compute the angle to the goal, set a desired velocity of magnitude *speed* in that

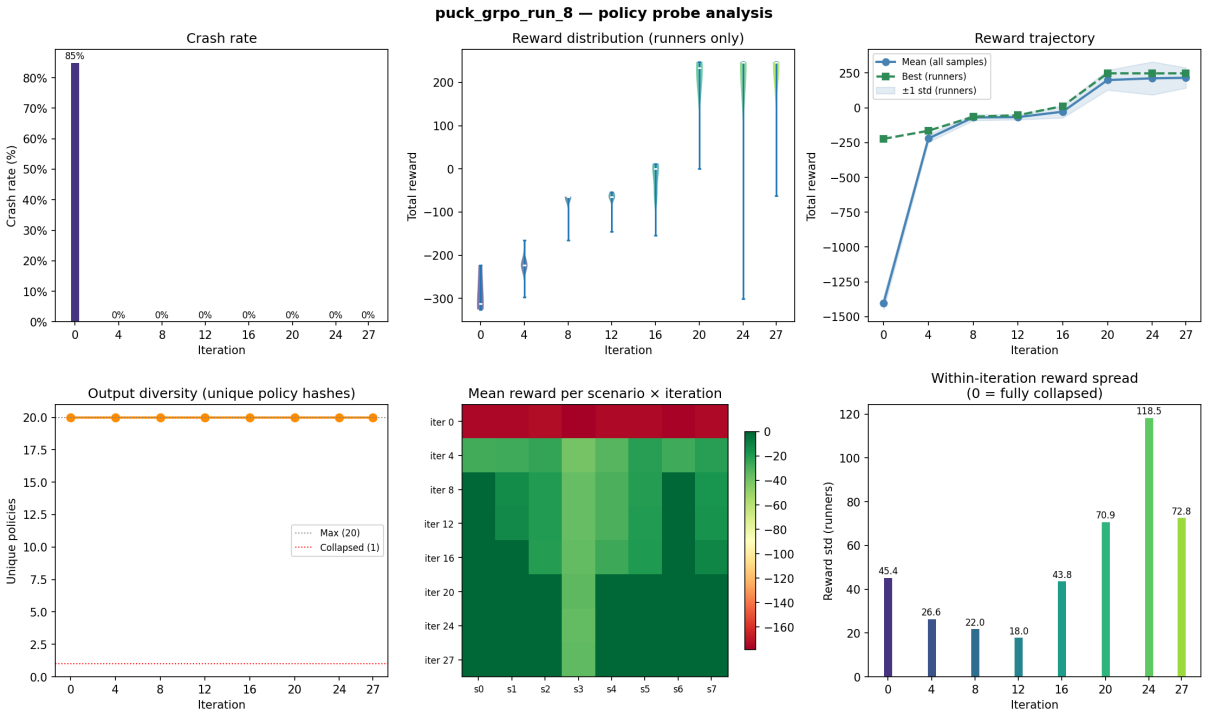


Figure 2: Run 8 learning curve: mean and best reward across 8 probed iterations ($n = 20$ samples each). Generated by `scripts/analyze_probe.py --run_name puck_grpo_run_8`.

direction, and command acceleration proportional to the velocity error:

$$a = \text{gain} \times (v_{\text{desired}} - v_{\text{current}})$$

The only variation across the 20 samples is in two scalar hyperparameters: desired speed (1.0–5.0) and gain ($/0.05 = \times 20$, $\times 10$, $\times 20$). The three below-average samples each have a distinct defect: sample 08 contains a positional-step bug that degenerates to naive aim (reward -62.8); sample 10 sets speed $= 1.0$, too slow to reach the goal in 120 steps; sample 15 self-clips at 3.0 below the env's own $a_{\max}$.

Iter 0 — API mismatch causes 85% crash rate. 17 of 20 base-model policies crashed with KeyError at reward floor -1600. The specific hidden keys accessed are summarised in Table 2. The three survivors avoided hidden keys only by accident: two hardcoded thrust magnitude constants and one hardcoded all physics parameters (`tilt_x = 0.0`, `mu = 0.5`, `a_max = 1.5`). The survivors scored -223, -312, and -325 — all worse than the naive aim baseline.

Hidden key	Samples accessing it
`a_max`	00, 01, 02, 03, 04, 05, 07, 08, 11, 12, 14, 15, 16, 19
`tilt_x`, `tilt_y`	01, 03, 04, 06, 08, 11, 12, 13, 14, 15, 16, 19
`mu`	01, 04, 07, 08, 12, 15, 16
`dt`	16, 17

Table 2: Hidden state keys accessed by crashing iter-0 samples.

Interpretation

What iter 27 discovered. The trained model found a velocity P-controller rather than the PD-on-position + tilt-feedforward strategy used by the hand-written baseline. This is structurally

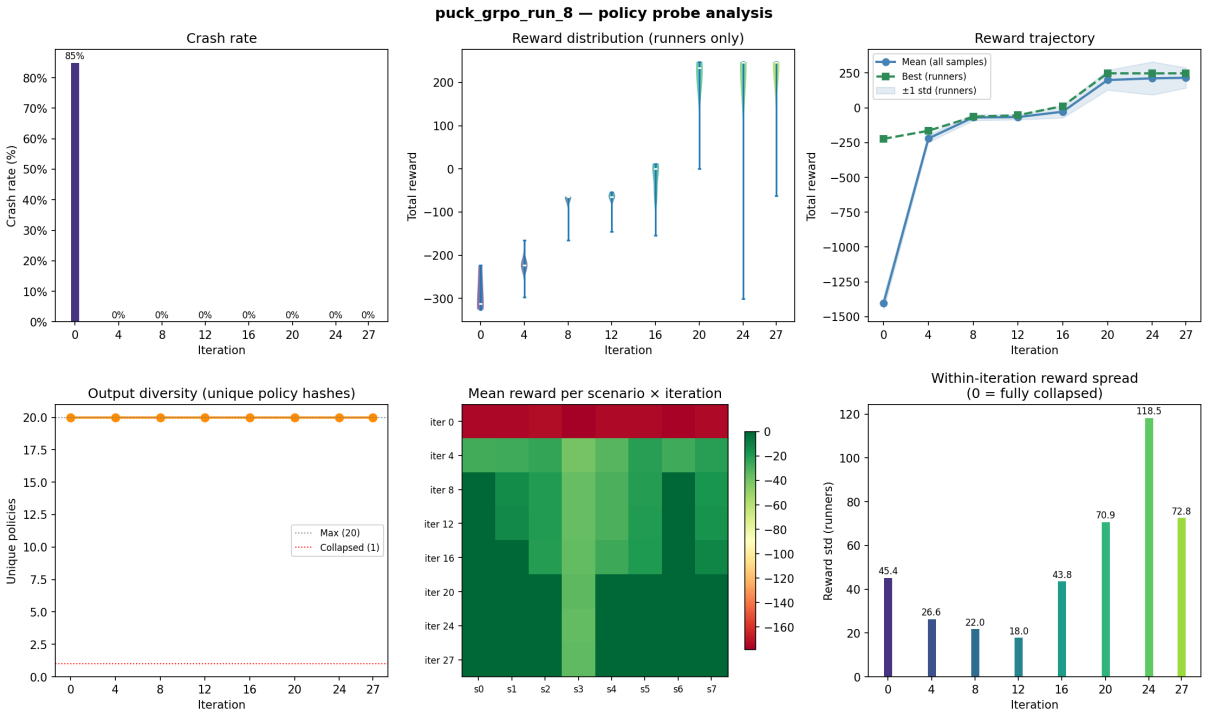


Figure 3: Run 8 learning curve across the 8 probed iterations, reproduced here for context. The phase transition at iter 16–20 is the backdrop for the policy structure analysis above.

different: there is no position error term and no tilt awareness. The approach works because high-gain velocity tracking rejects tilt disturbances reactively within a few timesteps, yielding near-ceiling reward without explicit feedforward. The residual 8% gap to the hand-written ceiling (+248 vs. ≈ +270) is consistent with reactive rather than proactive tilt rejection.

Why iter 0 failed. The base model's prior — drawn from pretraining on standard RL codebases — is that a physics simulation exposes all its parameters through the state dict or environment attributes. 14 of 17 crashing samples attempted tilt compensation or drag cancellation that would have been correct if the hidden parameters were accessible. The model's control-theory knowledge was largely sound; its environment-API assumption was wrong. GRPO fixed the API assumption without teaching better control theory, because velocity tracking was sufficient to saturate the available reward.

Hypotheses / next steps

1. **Probe the transition from position-thrust (iter 0 survivors) to velocity-tracking (iter 27).** At which iteration does the velocity-control template first appear? Checking iter-4 policies would show whether it emerges immediately after crash rate drops.
2. **Test whether harder tilt distributions force tilt compensation.** With wider tilt ranges the reactive velocity-tracker may fail; GRPO would then need to discover an explicit feedforward term to push reward higher.
3. **Provide the hidden parameters in the state dict and retrain.** If the base model already knows how to use tilt/drag/a_max (most iter-0 samples attempt it), a single run with those keys exposed could achieve the hand-written ceiling immediately and reveal what further headroom exists beyond it.

Open questions

- Would the velocity-tracking template generalise to a new environment with different dynamics, or is it a memorised pattern for the puck task?
- The best iter-27 policy (+247.6) and the best iter-0 survivor (−223) both use only valid state keys. Is the iter-27 template a refinement of the iter-0 survivor template, or did it emerge independently under GRPO pressure?

Reproduce

Manual inspection at `e6bc200`; policy files already present in `data/probe/puck_grpo_run_8_iter{000,027}_t0p8`